

CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

QUESTIONS

Q1
Student Management System
(BL3 – Apply)
10 marks**Q2**
2. Library Book Management
(BL3 – Apply)
10 marks**Q3**
Employee Salary Calculation
(BL3 – Apply)
10 marks**Q4**
Shape Area Calculator using
Polymorphism (BL3 – Apply)
10 marks**Q5**
Vehicle Registration System
(BL3 – Apply)
10 marks**Q6**
Bank Account Hierarchy (BL4 –
Analyse)
10 marks**Q7**
Hospital Patient Record System
(BL4 – Analyse)
10 marks**Q8**
E-Commerce Product Catalog
(BL4 – Analyse)
10 marks**Q9**
University Staff Hierarchy (BL4
– Analyse)
10 marks**Q10**
Smart Home Device Controller
(BL4 – Analyse)
10 marks

10/10 answered

Q1 Student Management System (BL3 – Apply)

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
CLASS Student
// Data Hiding: Declaring attributes as private
PRIVATE name AS STRING
PRIVATE rollNo AS INTEGER
PRIVATE marks AS ARRAY OF REAL // Array to hold marks for multiple subjects

// Constructor to initialize variables
PUBLIC CONSTRUCTOR()
    name = ""
    rollNo = 0
    // Initialize an empty list or array for marks
END CONSTRUCTOR

// Method to accept student details
PUBLIC METHOD acceptDetails()
    DISPLAY "Enter Student Name:"
    INPUT name
    DISPLAY "Enter Roll Number:"
    INPUT rollNo

    DISPLAY "Enter number of subjects:"
    INPUT numSubjects

    FOR i FROM 1 TO numSubjects
        DISPLAY "Enter marks for subject ", i, ":"
        INPUT m
        APPEND m TO marks
    ENDFOR
END METHOD

// Method to calculate average marks
PUBLIC METHOD calculateAverage() RETURNS REAL
    IF LENGTH(marks) == 0 THEN
        RETURN 0.0
    ENDIF

    total = 0.0
    FOR EACH mark IN marks
        total = total + mark
    ENDFOR

    average = total / LENGTH(marks)
    RETURN average
END METHOD

// Method to display result (Pass/Fail)
PUBLIC METHOD displayResult()
    avgMarks = calculateAverage()

    DISPLAY "--- Student Result ---"
    DISPLAY "Name: ", name
    DISPLAY "Roll No: ", rollNo
    DISPLAY "Average Marks: ", avgMarks

    // Assuming a minimum passing average of 50%
    IF avgMarks >= 50 THEN
        DISPLAY "Status: PASS"
    ELSE
        DISPLAY "Status: FAIL"
    ENDIF
END METHOD
END CLASS

// Main Program Execution (Object instantiation)
START
    CREATE studentObj AS Student
    studentObj.acceptDetails()
    studentObj.displayResult()
END
```

CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

QUESTIONS

Q1
Student Management System
(BL 1 - Apply)
10 marks**Q2**
2. Library Book Management
(BL 1 - Apply)
10 marks**Q3**
Employee Salary Calculation
(BL 1 - Apply)
10 marks**Q4**
Shape Area Calculation using
Polymorphism (BL 1 - Apply)
10 marks**Q5**
Vehicle Registration System
(BL 1 - Apply)
10 marks**Q6**
Bank Account Hierarchy (BL 4
- Analyze)
10 marks**Q7**
Hospital Patient Record System
(BL 4 - Analyze)
10 marks**Q8**
E-commerce Product Catalog
(BL 4 - Analyze)
10 marks**Q9**
University Staff Hierarchy (BL 4
- Analyze)
10 marks**Q10**
Smart Home Device Controller
(BL 4 - Analyze)
10 marks

10/10 answered

Q2. Library Book Management (BL1 - Apply)

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.
OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

```
METHOD getISBN()
    RETURN this.ISBN
END METHOD

METHOD getisAvailable()
    RETURN this.isAvailable
END METHOD

// Setter method for availability status
METHOD setisAvailable(status: Boolean)
    this.isAvailable = status
END METHOD

// Method to issue a book
METHOD issueBook()
    IF this.isAvailable == True THEN
        this.isAvailable = False
        PRINT "Book " + this.title + " has been successfully issued."
    ELSE
        PRINT "Sorry, " + this.title + " is currently not available."
    END IF
END METHOD

METHOD returnBook()
    IF this.isAvailable == False THEN
        this.isAvailable = True
        PRINT "Book " + this.title + " has been successfully returned."
    ELSE
        PRINT "Book " + this.title + " was not issued."
    END IF
END METHOD

// Method to display book details using getters
METHOD displayBookDetails()
    PRINT "--- Book Details ---"
    PRINT "Title: " + getTitle()
    PRINT "Author: " + getAuthor()
    PRINT "ISBN: " + getISBN()
    PRINT "Available: " + getisAvailable()
    PRINT "-----"
END METHOD

END CLASS

// Main Simulation Program
START
// Step 1: Create 3 book objects
CREATE book1 AS NEW Book("The Great Gatsby", "F. Scott Fitzgerald", "9780743273565")
CREATE book2 AS NEW Book("1984", "George Orwell", "9780451524935")
CREATE book3 AS NEW Book("To Kill a Mockingbird", "Harper Lee", "9780061120084")

// Step 2: Display initial details of the books
PRINT "Initial Book Statuses:"
book1.displayBookDetails()
book2.displayBookDetails()
book3.displayBookDetails()

// Step 3: Simulate issue operations
PRINT "\nSimulating Book Issuing:"
book1.issueBook() // Should succeed
book1.issueBook() // Should fail (already issued)
book2.issueBook() // Should succeed

// Step 4: Display status after issuing
PRINT "\nStatus After Issuing:"
book1.displayBookDetails()

// Step 5: Simulate Return operations
PRINT "\nSimulating Book Returning:"
book1.returnBook() // Should succeed
book3.returnBook() // Should show it wasn't issued

// Step 6: Final status check
PRINT "\nFinal Status of Book 1:"
book1.displayBookDetails()

END
```

[Save Answer](#)

CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

QUESTIONS

Q1
Student Management System
(BL3 – Apply)
10 marks

Q2
2. Library Book Management
(BL3 – Apply)
10 marks

Q3
Employee Salary Calculation
(BL3 – Apply)
10 marks

Q4
Shape Area Calculator using
Polymorphism (BL3 – Apply)
10 marks

Q5
Vehicle Registration System
(BL3 – Apply)
10 marks

Q6
Bank Account Hierarchy (BL4 –
Analyse)
10 marks

Q7
Hospital Patient Record System
(BL4 – Analyse)
10 marks

Q8
E-Commerce Product Catalog
(BL4 – Analyse)
10 marks

Q9
University Staff Hierarchy (BL4
– Analyse)
10 marks

Q10
Smart Home Device Controller
(BL4 – Analyse)
10 marks

10/10 answered

Q3 Employee Salary Calculation (BL3 – Apply)

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empld, name, basicPay, designation). Implement methods to: +
Calculate gross salary (basic + HRA + DA)
• Calculate net salary after deductions (PF, tax)
• Display pay slip
Apply data hiding so salary components are not directly accessible.
OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```
CLASS Employee
// Encapsulated / Private Attributes (Data Hiding)
PRIVATE empld: Integer
PRIVATE name: String
PRIVATE designation: String
PRIVATE basicPay: Real

// Constructor to initialize attributes
CONSTRUCTOR(id: Integer, empName: String, empDesignation: String, pay: Real)
    empld = id
    name = empName
    designation = empDesignation
    basicPay = pay
END CONSTRUCTOR

// Method to calculate gross salary
METHOD calculateGrossSalary() RETURNS Real
// Standard assumptions for allowances (e.g., HRA = 20%, DA = 10% of basic)
DECLARE hra: Real = basicPay * 0.20
DECLARE da: Real = basicPay * 0.10
RETURN basicPay + hra + da
END METHOD

// Method to calculate net salary
METHOD calculateNetSalary() RETURNS Real
DECLARE gross: Real = calculateGrossSalary()
// Standard assumptions for deductions (e.g., PF = 12%, Tax = 10% of gross)
DECLARE pf: Real = basicPay * 0.12
DECLARE tax: Real = gross * 0.10
RETURN gross - (pf + tax)
END METHOD

// Method to display pay slip
METHOD displayPaySlip()
DECLARE gross: Real = calculateGrossSalary()
DECLARE net: Real = calculateNetSalary()

PRINT "===== EMPLOYEE PAY SLIP ====="
PRINT "Employee ID: " + empld
PRINT "Name: " + name
PRINT "Designation: " + designation
PRINT "-----"
PRINT "Basic Pay: " + basicPay
PRINT "Gross Salary: " + gross
PRINT "Net Salary: " + net
PRINT "===== "
END METHOD
END CLASS
```

[Save Answer](#)

CO1-AT3-PSEUDOCODE WRITING

[← Back](#)

QUESTIONS

Q1
Student Management System
(BL3 – Apply)
10 marks**Q2**
Library Book Management
(BL3 – Apply)
10 marks**Q3**
Employee Salary Calculation
(BL3 – Apply)
10 marks**Q4**
Shape Area Calculator using
Polymorphism (BL3 – Apply)
10 marks**Q5**
Vehicle Registration System
(BL3 – Apply)
10 marks**Q6**
Bank Account Hierarchy (BL4 –
Analyse)
10 marks**Q7**
Hospital Patient Record System
(BL4 – Analyse)
10 marks**Q8**
E-Commerce Product Catalog
(BL4 – Analyse)
10 marks**Q9**
University Staff Hierarchy (BL4
– Analyse)
10 marks**Q10**
Smart Home Device Controller
(BL4 – Analyse)
10 marks

10/10 answered

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.
OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
CLASS Shape
METHOD calculateArea()
RETURN 0.0
END METHOD
END CLASS
```

```
CLASS Circle EXTENDS Shape
VARIABLES: radius
```

```
CONSTRUCTOR(r)
radius = r
END CONSTRUCTOR
```

```
OVERRIDE METHOD calculateArea()
RETURN 3.14159 * radius * radius
END METHOD
END CLASS
```

```
CLASS Rectangle EXTENDS Shape
VARIABLES: length, breadth
```

```
CONSTRUCTOR(l, b)
length = l
breadth = b
END CONSTRUCTOR
```

```
OVERRIDE METHOD calculateArea()
RETURN length * breadth
END METHOD
END CLASS
```

```
CLASS Triangle EXTENDS Shape
VARIABLES: base, height
```

```
CONSTRUCTOR(b, h)
base = b
height = h
END CONSTRUCTOR
```

```
OVERRIDE METHOD calculateArea()
RETURN 0.5 * base * height
END METHOD
END CLASS
```

// Main execution demonstrating runtime polymorphism

START

// Create an array/list of Shape references
Shape shapes[3]

// Instantiate child objects and assign them to the parent reference array

shapes[0] = NEW Circle(5.0)
shapes[1] = NEW Rectangle(4.0, 6.0)
shapes[2] = NEW Triangle(3.0, 8.0)

// Iterate through the array to demonstrate runtime polymorphism

FOR EACH shape IN shapes
PRINT "The area is: " + shape.calculateArea()
END FOR
END[Save Answer](#)

CO1-AT3-PSEUDOCODE WRITING

← Back

QUESTIONS

Q1
Student Management System
(B1.1 - Apply)
10 marks**Q2**
Library Book Management
(B1.1 - Apply)
10 marks**Q3**
Employee Salary Calculation
(B1.1 - Apply)
10 marks**Q4**
Diagon Sum Calculation using
Polymorphism (B1.1 - Apply)
10 marks**Q5**
Vehicle Registration System
(B1.1 - Apply)
10 marks**Q6**
Bank Account Withdrawal (B1.4 - Analyze)
10 marks**Q7**
Hospital Patient Record System
(B1.4 - Analyze)
10 marks**Q8**
E-commerce Product Catalog
(B1.4 - Analyze)
10 marks**Q9**
University Staff Hierarchy (B1.4 - Analyze)
10 marks**Q10**
Smart Home Device Controller
(B1.4 - Analyze)
10 marks

10/10 answered

Q5 Vehicle Registration System (B1.1 - Apply)

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.
OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

```
// Base Class
CLASS Vehicle
// Attributes
DECLARE regNo AS String
DECLARE owner AS String
DECLARE fuelType AS String

// Constructor
CONSTRUCTOR (regNo, owner, fuelType)
THIS.regNo = regNo
THIS.owner = owner
THIS.fuelType = fuelType
END CONSTRUCTOR

// Method to be overridden
METHOD displayDetails()
PRINT "Registration Number: " + THIS.regNo
PRINT "Owner: " + THIS.owner
PRINT "Fuel Type: " + THIS.fuelType
END METHOD
END CLASS

// Subclass for Two Wheelers
CLASS TwoWheeler EXTENDS Vehicle
// Additional Attribute
DECLARE engineCapacity AS Integer

// Constructor calling the base constructor
CONSTRUCTOR (regNo, owner, fuelType, engineCapacity)
SUPER (regNo, owner, fuelType)
THIS.engineCapacity = engineCapacity
END CONSTRUCTOR

// Overriding the base method
OVERRIDE METHOD displayDetails()
SUPER.displayDetails() // Reuses base class attributes printing
PRINT "Engine Capacity: " + THIS.engineCapacity + " cc"
END METHOD
END CLASS

// Subclass for Four Wheelers
CLASS FourWheeler EXTENDS Vehicle
// Additional Attributes
DECLARE seatingCapacity AS Integer
DECLARE hasAC AS Boolean

// Constructor calling the base constructor
CONSTRUCTOR (regNo, owner, fuelType, seatingCapacity, hasAC)
SUPER (regNo, owner, fuelType)
THIS.seatingCapacity = seatingCapacity
THIS.hasAC = hasAC
END CONSTRUCTOR

// Overriding the base method
OVERRIDE METHOD displayDetails()
SUPER.displayDetails() // Reuses base class attributes printing
PRINT "Seating Capacity: " + THIS.seatingCapacity
PRINT "Air Conditioning: " + (THIS.hasAC ? "Yes" : "No")
END METHOD
END CLASS

// Execution / Implementation Example
START
// Creating instances of subclasses
DECLARE bike AS TwoWheeler = NEW TwoWheeler("MH12-AB-1234", "John Doe", "Petrol", 150)
DECLARE car AS FourWheeler = NEW FourWheeler("DLJC-XY-5678", "Jane Smith", "Diesel", 5, TRUE)

// Demonstrating method overriding
PRINT "--- Two Wheeler Details ---"
bike.displayDetails()

PRINT "\n--- Four Wheeler Details ---"
car.displayDetails()
END
```

Save Answer

CO1-AT3-PSEUDOCODE WRITING

← Back

QUESTIONS

Q1
Student Management System
(BL 1 - Apply)
10 marks**Q2**
Library Book Management
(BL 1 - Apply)
10 marks**Q3**
Employee Salary Calculation
(BL 1 - Apply)
10 marks**Q4**
Shape Area Calculation using
Polymorphism (BL 1 - Apply)
10 marks**Q5**
Vehicle Registration System
(BL 1 - Apply)
10 marks**Q6**
Bank Account Hierarchy (BL 4
- Analyze)
10 marks**Q7**
Hospital Patient Record System
(BL 4 - Analyze)
10 marks**Q8**
E-commerce Product Catalog
(BL 4 - Analyze)
10 marks**Q9**
University Staff Hierarchy (BL 4
- Analyze)
10 marks**Q10**
Smart Home Device Controller
(BL 4 - Analyze)
10 marks

10/10 answered

Q6 Bank Account Hierarchy (BL4 - Analyze)

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.
OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```
CLASS BankAccount
// Inheritance of common attributes
PROTECTED accNo: String
PROTECTED balance: Double
PROTECTED holder: String

CONSTRUCTOR(accNo, holder, initialBalance)
THIS.accNo = accNo
THIS.holder = holder
THIS.balance = initialBalance
END CONSTRUCTOR

METHOD deposit(amount)
IF amount > 0 THEN
    THIS.balance = THIS.balance + amount
END IF
END METHOD

// Base withdrawal method to be overridden
METHOD withdraw(amount)
IF amount > 0 AND THIS.balance >= amount THEN
    THIS.balance = THIS.balance - amount
    RETURN TRUE
END IF
RETURN FALSE
END METHOD
END CLASS

CLASS SavingsAccount EXTENDS BankAccount
PRIVATE interestRate: Double // e.g., 0.04 for 4%

CONSTRUCTOR(accNo, holder, initialBalance, interestRate)
SUPER(accNo, holder, initialBalance)
THIS.interestRate = interestRate
END CONSTRUCTOR

METHOD calculateInterest()
RETURN THIS.balance * THIS.interestRate
END METHOD

METHOD addInterest()
THIS.balance = THIS.balance + THIS.calculateInterest()
END METHOD

// Method overriding for savings-specific withdrawal behavior
METHOD withdraw(amount) OVERRIDE
// Standard rule: Cannot withdraw below zero balance
IF amount > 0 AND THIS.balance >= amount THEN
    SUPER.withdraw(amount)
    RETURN TRUE
END IF
RETURN FALSE
END METHOD
END CLASS

CLASS CurrentAccount EXTENDS BankAccount
PRIVATE overdraftLimit: Double // Maximum allowed negative balance

CONSTRUCTOR(accNo, holder, initialBalance, overdraftLimit)
SUPER(accNo, holder, initialBalance)
THIS.overdraftLimit = overdraftLimit
END CONSTRUCTOR

// Method overriding for overdraft-specific withdrawal behavior
METHOD withdraw(amount) OVERRIDE
// Allows balance to go negative down to the overdraft limit
IF amount > 0 AND (THIS.balance + THIS.overdraftLimit) >= amount THEN
    THIS.balance = THIS.balance - amount
    RETURN TRUE
END IF
RETURN FALSE
END METHOD
END CLASS
```

Save Answer

CO1-AT3-PSEUDOCODE WRITING

← Back

QUESTIONS

- Q1** Student Management System (B1.1 - Apply) 10 marks
- Q2** Library Book Management (B1.1 - Apply) 10 marks
- Q3** Prescription Salary Calculation (B1.1 - Apply) 10 marks
- Q4** Diaper Area Calculation using Polymorphism (B1.1 - Apply) 10 marks
- Q5** Vehicle Registration System (B1.1 - Apply) 10 marks
- Q6** Bank Account Minus by (B1.4 - Analyze) 10 marks
- Q7** Hospital Patient Record System (B1.4 - Analyze) 10 marks
- Q8** E-commerce Product Catalog (B1.4 - Analyze) 10 marks
- Q9** University Staff Minus by (B1.4 - Analyze) 10 marks
- Q10** Smart Home Device Controller (B1.4 - Analyze) 10 marks

10/10 answered

Q7 Hospital Patient Record System (B1.4 - Analyze)

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```

END CONSTRUCTOR

// Encapsulation: Controlled access via Getters and Setters
METHOD getPatientId()
    RETURN THIS.patientId
END METHOD

METHOD getName()
    RETURN THIS.name
END METHOD

// Access restricted to authorized medical personnel role
METHOD getDiagnosis(userRole)
    IF userRole EQUALS "Doctor" OR userRole EQUALS "Nurse" THEN
        RETURN THIS.diagnosis
    ELSE
        RETURN "ACCESS DENIED: Unauthorized Role"
    END IF
END METHOD

METHOD setDiagnosis(newDiagnosis, userRole)
    IF userRole EQUALS "Doctor" THEN
        THIS.diagnosis = newDiagnosis
    ELSE
        PRINT "ACCESS DENIED: Only Doctors can modify diagnosis"
    END IF
END METHOD

METHOD getPrescription(userRole)
    IF userRole EQUALS "Doctor" OR userRole EQUALS "Pharmacist" THEN
        RETURN THIS.prescription
    ELSE
        RETURN "ACCESS DENIED: Unauthorized Role"
    END IF
END METHOD

// Polymorphic method base declaration
ABSTRACT METHOD generateBill()
// To be overridden by derived classes
END METHOD
END CLASS

// =====
// DERIVED CLASS: InPatient (Inherits Patient)
// =====
CLASS InPatient EXTENDS Patient
// Additional specific attributes
PRIVATE String wardNumber
PRIVATE String admissionDate
PRIVATE Double dailyCharges
PRIVATE Integer daysStayed

// Constructor calling super-class constructor
CONSTRUCTOR InPatient(id, name, diagnosis, prescription, wardNum, admDate, dailyChg, days)
    SUPER(id, name, diagnosis, prescription)
    THIS.wardNumber = wardNum
    THIS.admissionDate = admDate
    THIS.dailyCharges = dailyChg
    THIS.daysStayed = days
END CONSTRUCTOR

// Polymorphism: Overriding generateBill method
OVERRIDE METHOD generateBill()
    Double totalBill = THIS.dailyCharges * THIS.daysStayed
    PRINT "--- In-Patient Bill ---"
    PRINT "Patient ID: " + getPatientId()
    PRINT "Ward Number: " + THIS.wardNumber
    PRINT "Total Days: " + THIS.daysStayed
    PRINT "Total Accommodation Bill: $" + totalBill
    RETURN totalBill
END METHOD
END CLASS

// =====
// DERIVED CLASS: OutPatient (Inherits Patient)
// =====
CLASS OutPatient EXTENDS Patient

```

Save Answer

CO1-AT3-PSEUDOCODE WRITING

← Back

QUESTIONS

Q1 ✓
Student Management System
(R1.1 - Apply)
10 marks**Q2** ✓
Library Book Management
(R1.1 - Apply)
10 marks**Q3** ✓
Employee Salary Calculation
(R1.1 - Apply)
10 marks**Q4** ✓
Design Area Calculation using
Polymorphism (R1.1 - Apply)
10 marks**Q5** ✓
Vehicle Registration System
(R1.1 - Apply)
10 marks**Q6** ✓
Bank Account Interest by (R1.4 - Analyze)
10 marks**Q7** ✓
Hospital Patient Record System
(R1.4 - Analyze)
10 marks**Q8** ✓
E-Commerce Product Catalog
(R1.4 - Analyze)
10 marks**Q9** ✓
University Staff Bonus by (R1.4 - Analyze)
10 marks**Q10** ✓
Smart Home Device Controller
(R1.4 - Analyze)
10 marks

10/10 answered

E-Commerce Product Catalog (R1.4 - Analyze)

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```
// Base Class
CLASS Product
ATTRIBUTES:
    String name
    double price

CONSTRUCTOR Product(name, price)
    this.name = name
    this.price = price
END CONSTRUCTOR

// Polymorphic method to be overridden
METHOD applyDiscount()
    RETURN price // No discount by default
END METHOD
END CLASS

// Child Class: Electronics
CLASS Electronics INHERITS Product
ATTRIBUTES:
    int warrantyMonths
    String brand
    int voltage

CONSTRUCTOR Electronics(name, price, warrantyMonths, brand, voltage)
    SUPER(name, price)
    this.warrantyMonths = warrantyMonths
    this.brand = brand
    this.voltage = voltage
END CONSTRUCTOR

OVERRIDE METHOD applyDiscount()
    RETURN price * 0.90 // 10% flat discount on Electronics
END METHOD
END CLASS

// Child Class: Clothing
CLASS Clothing INHERITS Product
ATTRIBUTES:
    String size
    String fabric
    String gender

CONSTRUCTOR Clothing(name, price, size, fabric, gender)
    SUPER(name, price)
    this.size = size
    this.fabric = fabric
    this.gender = gender
END CONSTRUCTOR

OVERRIDE METHOD applyDiscount()
    RETURN price * 0.80 // 20% flat discount on Clothing
END METHOD
END CLASS

// Child Class: Grocery
CLASS Grocery INHERITS Product
ATTRIBUTES:
    Date expiryDate
    double weight

CONSTRUCTOR Grocery(name, price, expiryDate, weight)
    SUPER(name, price)
    this.expiryDate = expiryDate
    this.weight = weight
END CONSTRUCTOR

OVERRIDE METHOD applyDiscount()
    RETURN price * 0.95 // 5% flat discount on Grocery
END METHOD
END CLASS
```

Save Answer

CO1-AT3-PSEUDOCODE WRITING

4/3/23

QUESTIONS

Q1
Student Management System
(B.1 - Apply)
10 marks**Q2**
Library Book Management
(B.1 - Apply)
10 marks**Q3**
Employee Salary Calculation
(B.1 - Apply)
10 marks**Q4**
Voter Data Calculation using
Polymorphism (B.1 - Apply)
10 marks**Q5**
Vehicle Registration System
(B.1 - Apply)
10 marks**Q6**
Bank Interest Hierarchy (B.4 - Analyze)
10 marks**Q7**
Hospital Patient Record System
(B.4 - Analyze)
10 marks**Q8**
E-commerce Product Catalog
(B.4 - Analyze)
10 marks**Q9**
University Staff Hierarchy (B.4 - Analyze)
10 marks**Q10**
Smart Home Device Controller
(B.4 - Analyze)
10 marks

10/10 answered

University Staff Hierarchy (B.4 - Analyze)

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure. Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```
// =====  
// LEVEL 1: BASE CLASS  
// =====  
CLASS Staff  
    // Private: Accessible only within this class  
    PRIVATE String staffId  
    PRIVATE String name  
  
    // Protected: Accessible by child classes but hidden from public  
    PROTECTED Double baseSalary  
  
    CONSTRUCTOR Staff(id, staffName, salary)  
        staffId = id  
        name = staffName  
        baseSalary = salary  
    END CONSTRUCTOR  
  
    // Polymorphic method to be overridden  
    PUBLIC VIRTUAL FUNCTION calculateAllowance()  
        RETURN baseSalary * 0.10 // Default 10% base allowance  
    END FUNCTION  
END CLASS  
  
// =====  
// LEVEL 2: DERIVED CLASSES (Hierarchical)  
// =====  
CLASS TeachingStaff EXTENDS Staff  
    PRIVATE String researchDomain  
    PROTECTED Double academicAllowanceRate  
  
    CONSTRUCTOR TeachingStaff(id, staffName, salary, domain)  
        SUPER(id, staffName, salary) // Calls Staff constructor  
        researchDomain = domain  
        academicAllowanceRate = 0.15 // 15% special rate  
    END CONSTRUCTOR  
  
    OVERRIDE FUNCTION calculateAllowance()  
        // Accesses protected baseSalary directly from parent  
        RETURN baseSalary * academicAllowanceRate  
    END FUNCTION  
END CLASS  
  
CLASS NonTeachingStaff EXTENDS Staff  
    PRIVATE String department  
    PRIVATE Double overTimeHours  
  
    CONSTRUCTOR NonTeachingStaff(id, staffName, salary, dept, hours)  
        SUPER(id, staffName, salary)  
        department = dept  
        overTimeHours = hours  
    END CONSTRUCTOR  
  
    OVERRIDE FUNCTION calculateAllowance()  
        // Flat overtime rate calculation added to base allowance  
        RETURN (baseSalary * 0.05) + (overTimeHours * 25.0)  
    END FUNCTION  
END CLASS  
  
// =====  
// LEVEL 3: MULTILEVEL INHERITANCE  
// =====  
CLASS Professor EXTENDS TeachingStaff  
    PRIVATE String publicationCount  
  
    CONSTRUCTOR Professor(id, staffName, salary, domain, pubs)  
        SUPER(id, staffName, salary, domain)  
        publicationCount = pubs  
    END CONSTRUCTOR  
  
    OVERRIDE FUNCTION calculateAllowance()  
        // reuses base teaching staff calculation + adds bonus  
        RETURN SUPER.calculateAllowance() + 500.0  
    END FUNCTION  
END CLASS  
  
CLASS Lecturer EXTENDS TeachingStaff  
    PRIVATE Int teachingHours  
  
    CONSTRUCTOR Lecturer(id, staffName, salary, domain, hours)  
        SUPER(id, staffName, salary, domain)  
        teachingHours = hours  
    END CONSTRUCTOR  
  
    OVERRIDE FUNCTION calculateAllowance()  
        // Modified rate calculation using protected parent variables  
        RETURN baseSalary * (academicAllowanceRate + 0.05)  
    END FUNCTION  
END CLASS
```

Save Answer

CO1-AT3-PSEUDOCODE WRITING

Back

QUESTIONS

Q1
Busload Management System
(B1.1 - Apply)
10 marks**Q2**
Library Book Management
(B1.1 - Apply)
10 marks**Q3**
Employee Salary Calculation
(B1.1 - Apply)
10 marks**Q4**
Eigen Area Calculation using
Polymorphism (B1.1 - Apply)
10 marks**Q5**
Vehicle Registration System
(B1.1 - Apply)
10 marks**Q6**
Bank Account Hierarchy (B1.4 -
Analyze)
10 marks**Q7**
Hospital Patient Record System
(B1.4 - Analyze)
10 marks**Q8**
E-Commerce Product Catalog
(B1.4 - Analyze)
10 marks**Q9**
University Staff Hierarchy (B1.4 -
Analyze)
10 marks**Q10**
Smart Home Device Controller
(B1.4 - Analyze)
10 marks

10/10 answered

Q10 Smart Home Device Controller (B1.4 - Analyze)

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```
// =====  
// 1. SUPERCLASS DESIGN (Abstract Base)  
// =====  
ABSTRACT CLASS SmartDevice  
// Encapsulated common attributes  
PROTECTED String deviceId  
PROTECTED String deviceName  
PROTECTED String connectionStatus // e.g., "on", "off"  
  
// Constructor  
CONSTRUCTOR SmartDevice(id, name)  
this.deviceID = id  
this.deviceName = name  
this.connectionStatus = "off"  
END CONSTRUCTOR  
  
// Abstract method to be overridden by subclasses  
ABSTRACT METHOD executeCommand(String cmd)  
  
// Common utility method  
METHOD getStatus()  
RETURN this.deviceName + " (" + this.deviceID + ") is currently " + this.connectionStatus  
END METHOD  
END CLASS  
  
// =====  
// 2. SUBCLASS IMPLEMENTATIONS  
// =====  
  
CLASS SmartLight EXTENDS SmartDevice  
PRIVATE Integer brightnessLevel // unique attribute  
  
CONSTRUCTOR SmartLight(id, name)  
SUPER(id, name)  
this.brightnessLevel = 100  
END CONSTRUCTOR  
  
// Overriding Polymorphic Method  
OVERRIDE METHOD executeCommand(String cmd)  
IF cmd == "on" THEN  
this.connectionStatus = "on"  
PRINT this.deviceName + ": Light turned on at " + this.brightnessLevel + "% brightness."  
ELSEIF cmd == "off" THEN  
this.connectionStatus = "off"  
PRINT this.deviceName + ": Light turned off."  
ELSEIF cmd == "status" THEN  
PRINT SUPER.getStatus() + " | Brightness: " + this.brightnessLevel + "%"  
ELSE  
PRINT "Command not recognized by Light."  
END IF  
END METHOD  
END CLASS  
  
CLASS SmartThermostat EXTENDS SmartDevice  
PRIVATE Float temperature // unique attribute  
  
CONSTRUCTOR SmartThermostat(id, name)  
SUPER(id, name)  
this.temperature = 22.0 // default Celsius  
END CONSTRUCTOR  
  
OVERRIDE METHOD executeCommand(String cmd)  
IF cmd == "on" THEN  
this.connectionStatus = "on"  
PRINT this.deviceName + ": Climate control activated at " + this.temperature + "°C."  
ELSEIF cmd == "off" THEN  
this.connectionStatus = "off"  
PRINT this.deviceName + ": Climate control deactivated."  
ELSEIF cmd == "status" THEN  
PRINT SUPER.getStatus() + " | Target Temp: " + this.temperature + "°C"  
ELSE  
PRINT "Command not recognized by Thermostat."  
END IF  
END METHOD  
END CLASS
```

Save Answer